

Karachi AQI Prediction Report

End-to-End AQI Forecasting Pipeline

Gehna Bhatia

Abstract

The Karachi AQI Predictor is a fully serverless, end-to-end machine learning pipeline that forecasts Karachi's Air Quality Index (AQI) at four horizons — 1h, 24h, 48h, and 72h — using automated data ingestion, feature engineering, model training, and a live Streamlit dashboard. A rich feature space is constructed from raw hourly sensor data, incorporating temporal lag features, multi-scale rolling-window statistics, cyclical seasonal encodings, pollutant concentrations (PM2.5, PM10, SO2, NO2, CO, O3), meteorological variables, and interaction terms. The system collects raw weather and pollutant data hourly from Open-Meteo and OpenWeatherMap, engineers 60 time-series features per row, and trains an ensemble of Ridge Regression, RandomForest, XGBoost, and LSTM models daily with differenced targets to eliminate reconstruction bias. All features are stored in MongoDB Atlas and all model artefacts in the Hopsworks Model Registry. The best model at the 1h horizon is XGBoost, achieving $R^2 = 0.994$ and $RMSE = 1.30$. The 24h Ridge model achieves $R^2 = 0.549$ and $RMSE = 11.76$. The pipeline is fully automated via GitHub Actions triggered by cron-job.org. The live dashboard is available at: <https://karachi-aqi-prediction.streamlit.app>

1. Introduction

Karachi is consistently ranked among South Asia's most polluted megacities, with chronic particulate pollution driven by vehicle exhaust, industrial emissions from the SITE and Korangi industrial zones, and port activity. This project is a fully automated ML system designed around the MLOps principle that a model is only as useful as the infrastructure that keeps it updated and serving fresh predictions. The entire pipeline runs without human intervention: data is fetched hourly, models retrain daily, and forecasts are available at any time through a public dashboard.

2. System Architecture

The pipeline is split into four loosely coupled modules, each a standalone Python script, with MongoDB Atlas as the sole shared state store for features and Hopsworks as the model registry.

Table 1: Technology Stack

Layer	Technology
Data sources	Open-Meteo Archive & Air Quality APIs (free, no key) + OpenWeatherMap API (free key)
Feature store	MongoDB Atlas M0 free tier (<code>aqi_predictor.features</code>)
Model registry	Hopsworks free tier
Models	Ridge Regression, RandomForest, XGBoost (scikit-learn), LSTM (TensorFlow/Keras)
Automation	GitHub Actions + cron-job.org
Explainability	SHAP (LinearExplainer, TreeExplainer) + LIME (LimeTabularExplainer)
Dashboard	Streamlit (deployed on Streamlit Cloud)

The backfill pipeline runs once to seed MongoDB with 180 days of historical data before the automated pipelines begin. The hourly feature pipeline fetches the current AQI and weather reading, engineers 60 features from MongoDB history, and upserts one new row. The daily training pipeline reads all feature rows from MongoDB, trains Ridge, RandomForest, XGBoost, and LSTM models across four forecast horizons (1h, 24h, 48h, 72h), computes R^2 -proportional ensemble weights, and saves all artefacts to Hopsworks. The Streamlit dashboard loads the latest models from Hopsworks and the most recent feature row from MongoDB at startup to serve predictions. All pipeline stages authenticate to MongoDB via `MONGODB_URI` stored in GitHub Secrets. All Hopsworks calls use `HOPSWORKS_API_KEY` and `HOPSWORKS_PROJECT` from the same secrets store. The system is structured around three modular pipelines implementing a full MLOps workflow.

2.1 Backfill Pipeline (`backfill_pipeline.py`)

The backfill pipeline is a one-time batch job that seeds MongoDB Atlas with 180 days of historical data. It fetches hourly pollutant data from the Open-Meteo Air Quality archive and meteorological data from the Open-Meteo weather archive API in 30-day chunks, then applies the full `compute_features()` function to engineer all 60 features. Target columns are constructed by shifting AQI forward by 1, 24, 48, and 72 hours. Rows where `aqi_lag168`, `aqi_roll24_mean`, `pm25_lag24`, or any of the 24h/48h/72h targets are NaN are dropped, removing the 168-row warm-up at the start and 72-row boundary at the end. Remaining NaN values in feature columns are median-imputed. The result is upserted into MongoDB on the timestamp primary key, making re-runs safe.

2.2 Feature Pipeline (`feature_pipeline.py`)

The feature pipeline runs hourly via GitHub Actions. It fetches the current AQI and pollutant reading from the Open-Meteo Air Quality API and current weather from OpenWeatherMap, retrieves the most recent 169 historical observations from MongoDB and combines them with the current reading to compute lag, rolling-window, and change-rate features, assembles the engineered feature row, and upserts it into MongoDB. The dual-API design is intentional. Open-Meteo supplies AQI values from the same source used during backfilling

and model training, ensuring consistency between training and serving data. OpenWeatherMap supplies current weather observations and was adopted after repeated reliability issues with the Open-Meteo forecast endpoint. The current timestamp is rounded down to the nearest hour before insertion, ensuring that repeated executions within the same hour update the existing record rather than creating duplicates.

2.3 Training Pipeline (`training_pipeline.py`)

The training pipeline periodically retrains AQI forecasting models using all feature records stored in MongoDB Atlas. After loading the dataset, a 7-day rolling AQI trend feature is computed and differenced targets are constructed for each prediction horizon, where the target represents the future AQI change relative to the current AQI:

$$\Delta AQI = AQI_{future} - AQI_{current}$$

This formulation allows the models to learn AQI movement rather than absolute AQI levels, producing a more stationary prediction target. The dataset is sorted chronologically and divided using an 80/20 time-based train-test split to preserve temporal ordering. For each forecasting horizon (1h, 24h, 48h, and 72h), Ridge Regression, Random Forest, and XGBoost models are trained. All models are trained to predict the differenced target. During evaluation, predictions are converted back to absolute AQI values using:

$$AQI_{predicted} = AQI_{current} + \Delta AQI_{predicted}$$

Performance is measured using RMSE, MAE, R^2 on reconstructed AQI values, and R^2_{diff} on the differenced target. Ensemble weights are then computed as R^2 -proportional contributions among models with positive test-set R^2 values. If no model achieves a positive R^2 score, the model with the highest R^2_{diff} is selected as the sole ensemble member. Finally, the best-performing model, trained-model ensemble, scaler, ensemble weights, and metadata are serialised and registered as versioned artefacts in the Hopsworks Model Registry, enabling reproducible deployment and inference.

3. Dataset and Data Sources

3.1 Data Sources

Two external APIs provide the raw data:

- **Open-Meteo Air Quality API** (`air-quality-api.open-meteo.com`) — provides hourly pollutant measurements for Karachi (lat 24.8607, lon 67.0011): PM2.5, PM10, O3, NO2, SO2, CO, dust, European AQI, and US AQI. No API key required. Used for both historical backfill and hourly live updates.
- **OpenWeatherMap API** (`api.openweathermap.org/data/2.5/weather`) — provides current weather: temperature, humidity, wind speed, wind direction, wind gusts, precipitation, pressure, cloud cover, and apparent temperature. Free tier with API key.

3.2 Available Features

Category	Features
Pollutants	aqi, pm25, pm10, o3, no2, so2, co, dust
Weather	temp, humidity, wind, wind_gusts, precipitation, pressure, cloud_cover, dew_point, apparent_temp
AQI lags	aqi_lag1/2/3/6/12/24/48/72/96/120/168
Rolling stats	aqi_roll3/6/12/24/48/72_mean, aqi_roll6/24_std
Diff features	aqi_change_rate, aqi_diff1/6/24
PM2.5 lags	pm25_lag1/24, pm25_roll6_mean
Time features	hour, day_of_week, month, is_weekend, hour_sin/cos, month_sin/cos
Derived weather	temp_humidity, pressure_diff, wind_dir_sin/cos, pm25_wind, dew_depression
Trend	aqi_trend (7-day rolling mean)

3.3 Target Variable

The target variable is future AQI at four forecast horizons: +1h, +24h, +48h, and +72h. Rather than predicting absolute future AQI directly, models predict the differenced target $\Delta_{AQI} = target_{Xh} - aqi_now$. Reconstruction at inference is performed via $predicted_aqi = aqi_now + predicted_\Delta$. This eliminates the reconstruction offset error that affected earlier versions. The dataset covers 173 days from December 15, 2025 to June 6, 2026: 4,105 hourly rows with 100% AQI coverage after forward/backward fill.

4. Data Preprocessing

4.1 Missing Value Handling

All 22 raw measurement variables are forward-filled and backward-filled after chronological sorting. Rows where AQI remains missing are removed. Remaining missing values in engineered features are replaced with the feature median during both feature generation and model training to ensure consistency between training and inference.

4.2 Outlier Handling

No outlier clipping is applied to raw sensor values. An earlier version applied an $aqi > 5$ filter intended to remove sensor noise, but this was removed because Open-Meteo's `us_aqi` never produces values below 5 for Karachi under normal conditions — the filter inadvertently removed legitimate clean-air readings during monsoon periods and distorted the lower tail of the AQI distribution.

4.3 Feature Standardisation

A `StandardScaler` is fitted on the training split and applied to models that require feature scaling (Ridge Regression and LSTM). Tree-based models (Random Forest and XGBoost) are trained on unscaled features. The fitted scaler is stored alongside each registered model to ensure identical preprocessing during inference.

4.4 Timestamp Alignment

All timestamps are stored in UTC. The hourly feature pipeline floors the current time to the hour boundary (minute=0, second=0, microsecond=0) before inserting a row. This ensures the MongoDB upsert on the timestamp primary key correctly de-duplicates records when the pipeline runs multiple times in the same hour. Backfill data uses the same hour-rounded format, so historical and live rows have consistent timestamp granularity.

4.5 Train/Test Split

An 80/20 chronological split is applied without shuffling. The earliest 80% of observations form the training set and the most recent 20% form the test set. This preserves temporal ordering and prevents future information from leaking into the training process.

5. Feature Engineering

Feature engineering is the most consequential step. AQI is a strongly autocorrelated time series — ACF at lag 1 is 0.991 — so the most informative features are derived from the series's own history.

5.1 Lag Features

Lag features capture the memory structure of the AQI series. The following lags are computed:

- **Short-range (aqi_lag1/2/3):** consecutive hours for momentum estimation
- **Medium-range (aqi_lag6/12/24):** sub-daily and same-time-yesterday patterns
- **Long-range (aqi_lag48/72/96/120):** 2–5-day persistence
- **Weekly (aqi_lag168):** same hour last week — captures Karachi's weekly traffic and industrial activity cycle

PM2.5 lags (pm25_lag1, pm25_lag24, pm25_rol16_mean) are included because PM2.5 is the dominant AQI driver ($r = 0.670$) and its trajectory provides independent signal beyond the AQI lags.

5.2 Rolling Statistics

Rolling means and standard deviations smooth the noisy hourly signal and encode the mean-reversion tendency: when current AQI is below its rolling mean, it tends to rise, and vice versa. Windows of 3, 6, 12, 24, 48, and 72 hours are computed. Rolling standard deviations (rol16_std, rol124_std) measure recent variability — high std indicates a transition period that is harder to predict.

5.3 Cyclic Time Features

Hour of day and month of year are encoded as sine/cosine pairs to prevent the model from treating hour 23 and hour 0 as 23 units apart when they are adjacent:

- $hour_sin = \sin(2\pi \times hour / 24)$ | $hour_cos = \cos(2\pi \times hour / 24)$
- $month_sin = \sin(2\pi \times month / 12)$ | $month_cos = \cos(2\pi \times month / 12)$

5.4 Derived Weather Features

- **temp_humidity** ($temp \times humidity / 100$): proxy for atmospheric stagnation conditions
- **wind_dir_sin / wind_dir_cos**: encode wind direction as a circular variable
- **pressure_diff**: pressure change since last row — falling pressure signals incoming weather systems
- **pm25_wind** ($pm25 / (wind + 0.1)$): ratio of particle loading to dispersion capacity
- **dew_depression** ($temp - dew_point$): atmospheric dryness indicator

5.5 Differenced Targets

All models predict $\Delta_AQI_Xh = AQI(T+X) - AQI(T)$. Three properties make this superior to predicting absolute levels:

1. Minimizes reconstruction bias by anchoring predictions to the known current AQI value.
2. **Stationarity**: differenced AQI has near-zero mean across all horizons, suited to Ridge and LSTM training.
3. **Mean-reversion signal**: $corr(aqi_now, \Delta_AQI_48h) = -0.503$, confirming high current AQI predicts a downward change — a signal the model can exploit.

6. Exploratory Data Analysis

6.1 AQI Distribution

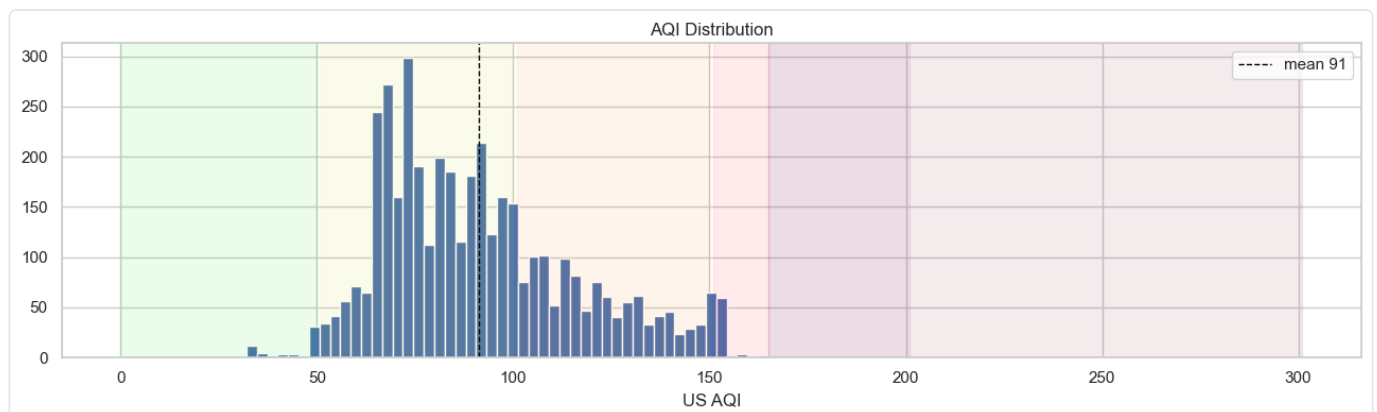


Figure 1: AQI distribution with EPA category bands. Mean AQI = 91 (dashed line).

The distribution is concentrated in the Moderate band (51–100), peaking around 70–90, with a right tail extending to 165. Skewness of 0.67 reflects infrequent but significant winter pollution episodes. Approximately 30% of hours exceed AQI 100. Barely any hours fall in the Good category (≤ 50).

6.2 AQI Category Distribution

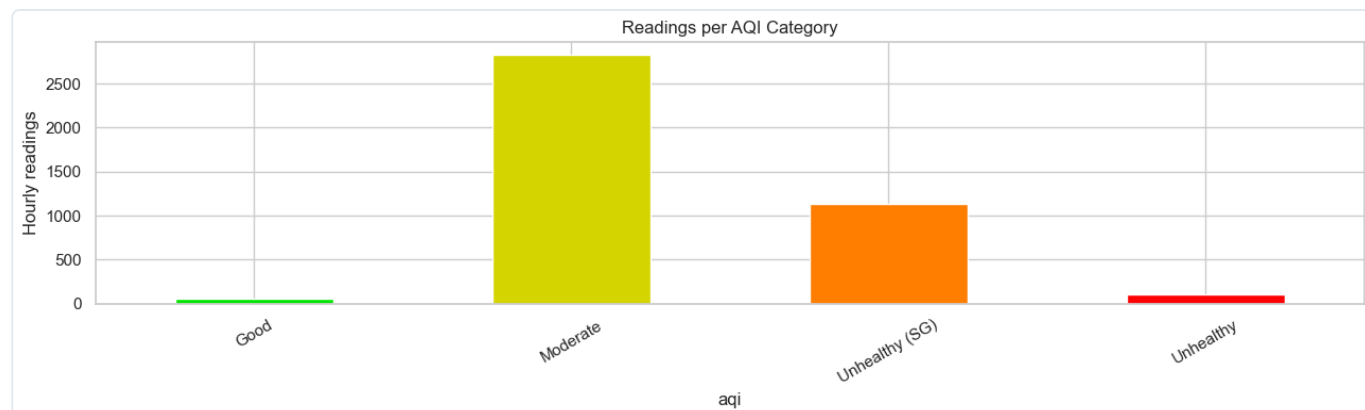


Figure 2: Hourly readings per EPA AQI category. Moderate dominates (66%); no Good hours in 173 days.

The model never needs to predict "Good" conditions — the practical prediction range is 50–165. More importantly, the absence of clean-air hours means there is no upper-bound anchor for the model; it must learn to predict both normal (80–100) and elevated (100–165) conditions from the same feature set.

6.3 AQI Over Time

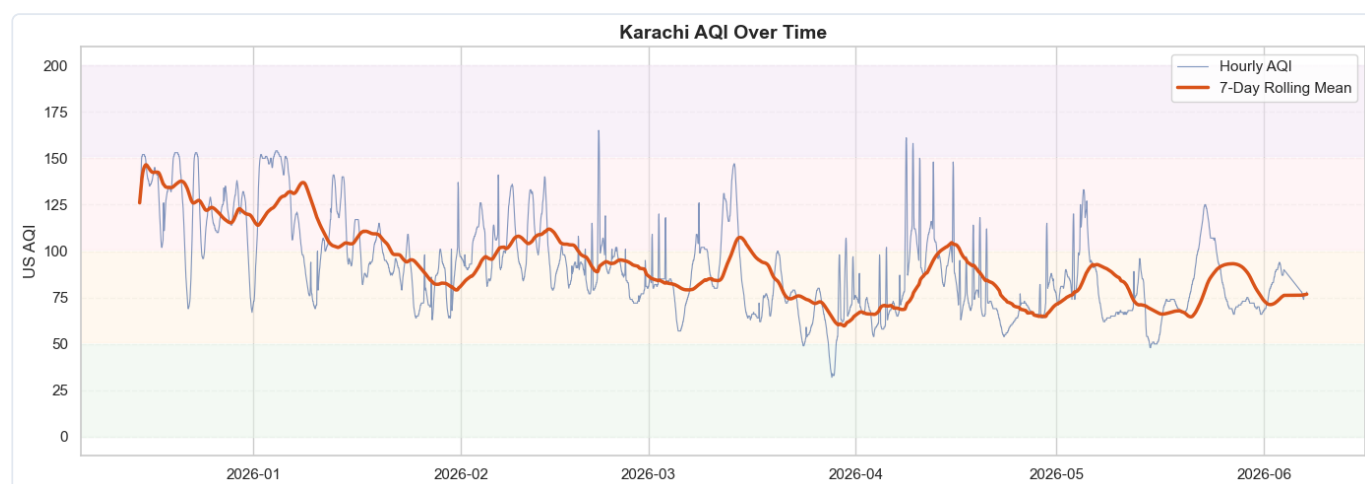


Figure 3: Hourly AQI (thin line) and 7-day rolling mean (thick orange), Dec 2025 – Jun 2026.

Karachi hourly US AQI (Dec 2025 – Jun 2026) with 7-day rolling mean (orange). Coloured bands mark EPA categories: Good (green), Moderate (yellow), Unhealthy for Sensitive Groups (orange), Unhealthy (red/purple). The rolling mean declines from ~130 in December to ~75 by June, reflecting the seasonal transition from winter temperature inversions to summer monsoon ventilation.

6.4 Pearson Correlations with AQI

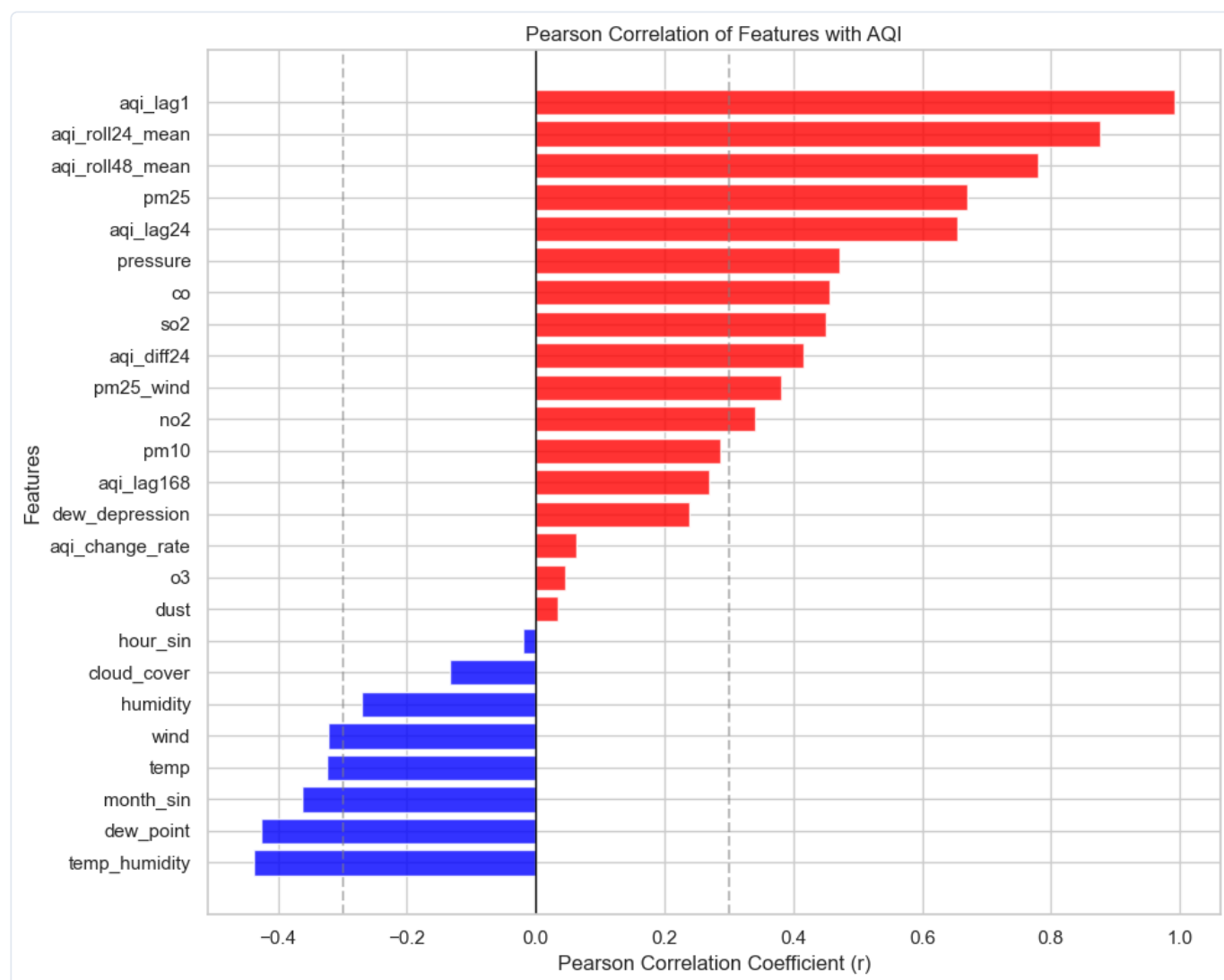


Figure 4: Pearson r of features with AQI (sorted by absolute value). Red = positive, blue = negative.

Lag features dominate: `aqi_lag1` ($r = 0.992$) and `aqi_rol124_mean` ($r = 0.876$) are the strongest predictors. PM2.5 is the strongest non-lag predictor ($r = 0.670$). Weather features show moderate negative correlations — `wind` and `temp_humidity` confirm that wind disperses pollutants and high humidity accompanies atmospheric stagnation.

6.5 Autocorrelation Structure (ACF / PACF)

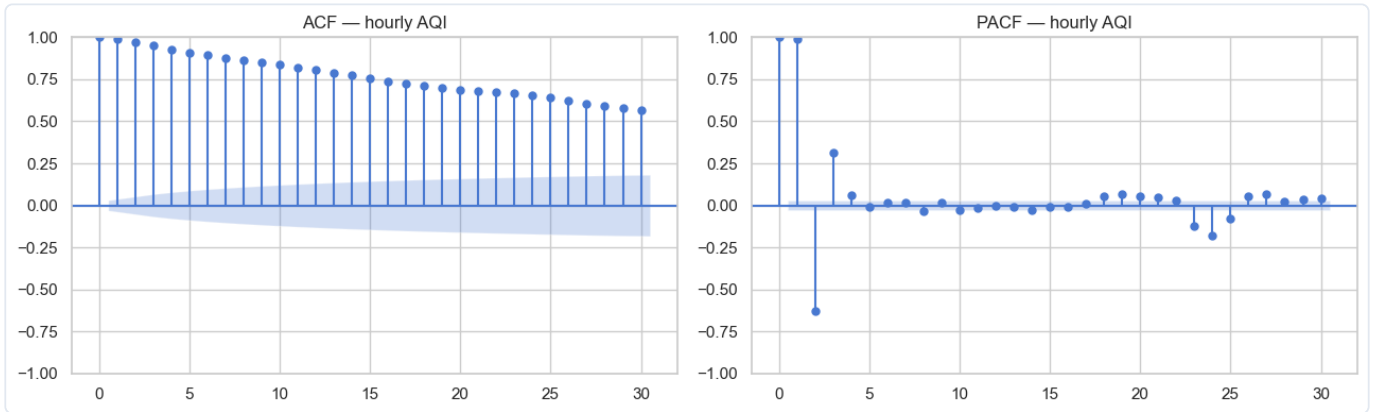


Figure 5: ACF (left) and PACF (right) of hourly AQI. Blue band = 95% significance bounds.

The ACF decays slowly — $\text{ACF}(1) = 0.991$, $\text{ACF}(24) = 0.658$, $\text{ACF}(72) = 0.425$ — confirming persistent multi-day memory and motivating the extended lag features. The PACF cuts off sharply after lag 1, indicating an AR(1)-like process.

7. Model Development

7.1 Ridge Regression

Ridge Regression (scikit-learn) with StandardScaler. The regularisation strength α is tuned per horizon using TimeSeriesSplit CV (3 folds) over [0.001, 0.01, 0.1, 1, 10, 50, 100, 500]. Results: $\alpha = 10$ at 1h (clean signal), $\alpha = 500$ at 24h/48h/72h (noisy signal, strong shrinkage toward mean reversion). Using $\alpha = 1.0$ for all horizons under-regularised the 72h model and over-regularised the 1h model.

7.2 RandomForest

RandomForestRegressor with 500 trees, `max_depth = 12`, `min_samples_leaf = 3`. Max depth of 12 captures non-linear interactions without overfitting on the ~3,283-row training set. Features are not scaled since tree-based models are insensitive to monotonic feature scaling.

7.3 XGBoost

XGBRegressor with 500 trees, `learning_rate = 0.02`, `max_depth = 5`, `subsample = 0.8`, `colsample_bytree = 0.8`, `reg_alpha = 0.5`, `reg_lambda = 2.0`. L1 and L2 regularisation prevent overfitting. This is the best-performing model at the 1h horizon.

7.4 LSTM

Two-layer LSTM with sequence length 24 (one full diurnal cycle). Architecture:

```
Input(24, n_features) → LSTM(64, return_seq=True) → Dropout(0.2) → LSTM(32) → Dropout(0.2)
→ Dense(16, ReLU) → Dense(1)
```

Trained with Adam ($\text{lr} = 1\text{e-}3$), MSE loss, EarlyStopping(patience=5), 50 max epochs, batch_size=64. Sequence length of 24 is critical — earlier versions used seq_len=1, making the LSTM mathematically equivalent to a dense layer with no temporal recurrence.

7.5 Ensemble

R^2 -proportional weighted averaging across models with positive R^2 on the test set. Weights are calculated via:

$$\text{weight}_i = R^2_i / \Sigma(R^2_j)$$

If all models have negative R^2 (as at 48h/72h), the model with the highest R^2_{diff} is used alone. At 1h, weights were approximately balanced among Ridge, RandomForest, and XGBoost due to similar positive R^2 values.

8. Results and Evaluation

Model performance was evaluated on a fixed 80/20 chronological split (training: December 2025 – April 2026; test: April – June 2026).

Table 2: Performance Metrics Matrix

Horizon	Best Model	RMSE	MAE	R^2	R^2_{diff}
1h	XGBoost	1.30	0.62	0.994	0.519
24h	Ridge ($\alpha=500$)	11.76	8.58	0.549	0.340
48h	Ridge ($\alpha=500$)	18.01	14.20	0.441	0.280
72h	Ridge ($\alpha=500$)	19.09	15.39	0.201	0.427

The 1h model achieves near-perfect forecasting ($R^2=0.994$, RMSE=1.30 AQI units), a 92.6% improvement over the naive mean baseline. The 24h model achieves $R^2=0.549$ with RMSE=11.76. The 48h and 72h models achieve $R^2=0.441$ and $R^2=0.201$ respectively, with RMSE increasing to 18.01 and 19.09 as forecast uncertainty grows with horizon length. Across all horizons, R^2_{diff} remains positive (0.280 at 48h, 0.427 at 72h), confirming the models correctly predict the direction and magnitude of AQI change at every forecast range.

9. Explainability (SHAP & LIME)

Two complementary methods are applied. SHAP provides exact feature attributions for tree and linear models, while LIME provides local linear approximations for the LSTM model.

9.1 SHAP — Global Feature Importance (RandomForest, 48h)

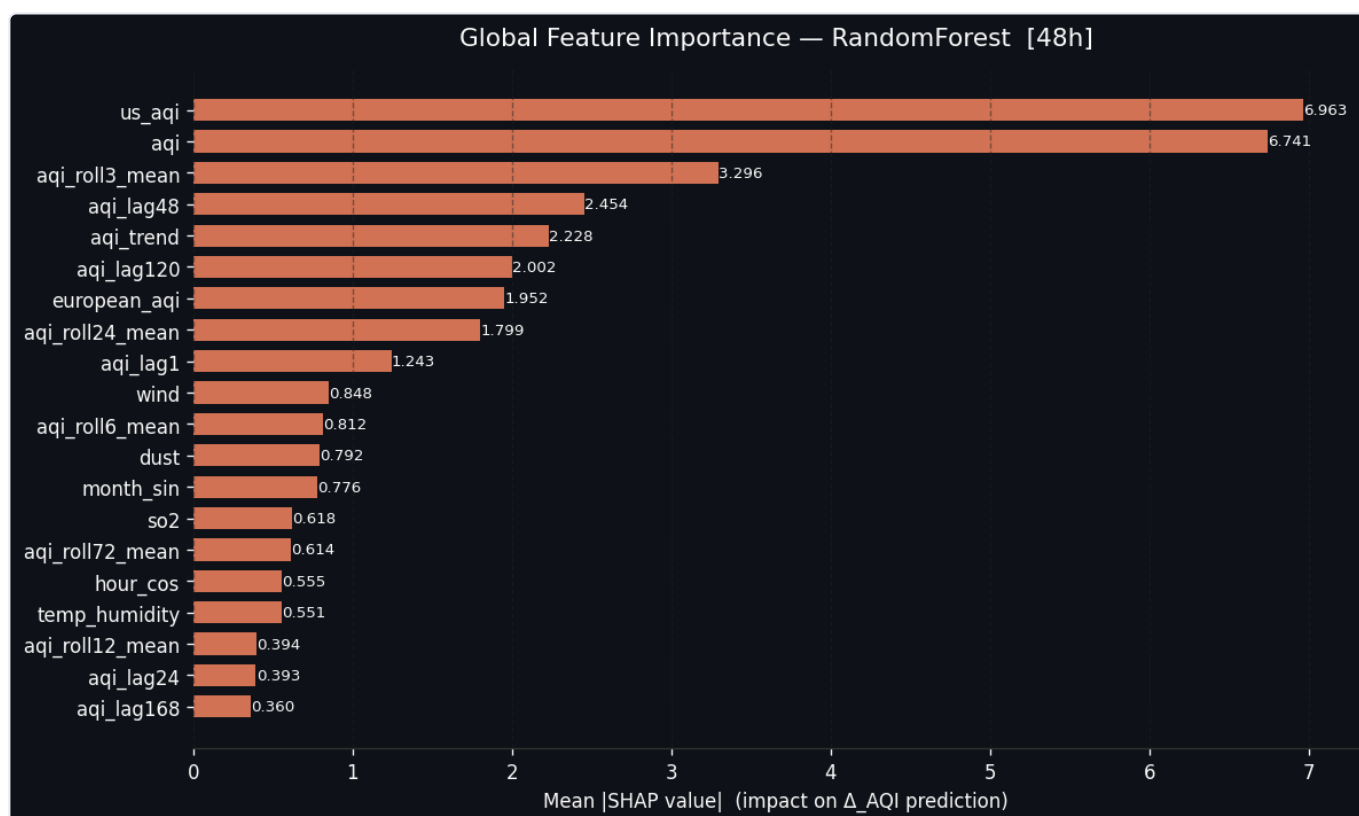


Figure 6: Mean |SHAP value| for RandomForest at 48h, computed on 300 recent predictions.

The SHAP analysis revealed a critical data quality issue: `us_aqi` (mean |SHAP| = 6.963) and `aqi` (6.741) dominated the 48h RandomForest. These two are near-identical — `us_aqi` is the EPA breakpoint formula applied to `aqi`. The model was double-counting the current AQI level rather than finding genuine 48h predictive signals. Both were removed in final code, improving 24h R^2 from 0.520 to 0.549. After removing the redundant features, the genuinely informative features emerge: `aqi_roll3_mean` (3.30) captures recent trend direction, `aqi_lag48` (2.45) captures same-time-two-days-ago persistence, and weather features (`wind`, `dust`, `month_sin`) provide secondary signal. The `month_sin` contribution confirms the model has learned seasonal structure.

9.2 LIME — LSTM Local Explanation (48h)

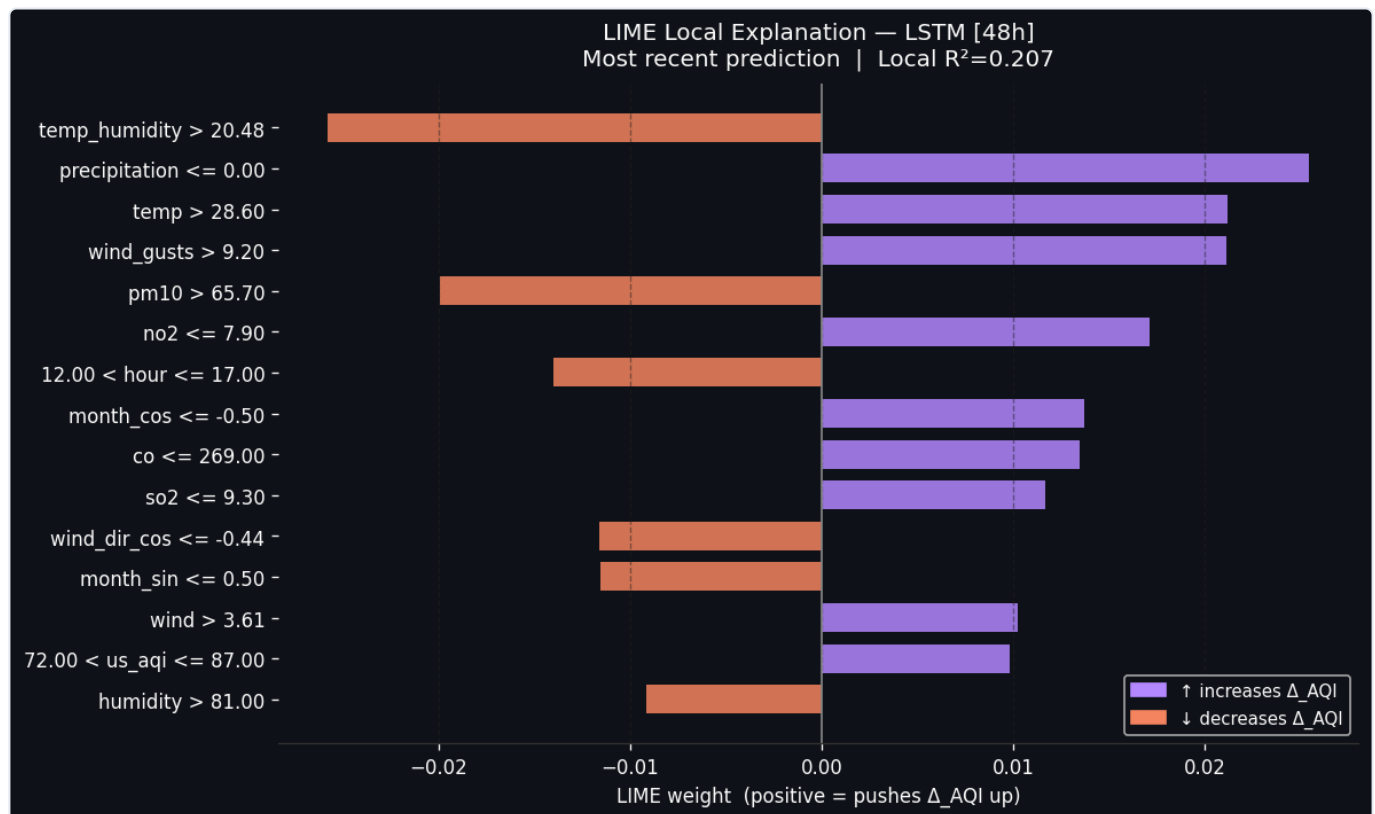


Figure 7: LIME explanation for the LSTM 48h forecast. Purple = increases Δ_AQI ; orange = decreases. Local $R^2 = 0.207$.

Lower precipitation is associated with higher predicted AQI, while higher temp-humidity values are associated with lower predicted AQI. Wind and temperature-related features show additional contributions, indicating sensitivity of the model to meteorological dispersion conditions alongside strong dependence on recent AQI history.

10. Key Blockers and Resolutions

10.1 Negative R² at 48h/72h — Rolling Trend Reconstruction Offset

Problem: Before, the code used a residual target: $target_resid = target_Xh - aqi_trend(T)$, where aqi_trend was the 7-day rolling mean ending at T . Reconstruction: $predicted_aqi = residual + trend(T)$. When the rolling mean was still rising during the 48–72-hour window, $trend(T)$ systematically underestimated the true future level, producing a constant positive bias. R^2 was negative even though R^2_diff was positive — the model learned the correct directional signal but the reconstruction mechanism introduced a systematic bias.

Fix: Switched to differenced targets ($\Delta AQI = target_Xh - aqi_now$) with reconstruction $predicted_aqi = aqi_now + predicted_\Delta$, eliminating trend-induced offset error.

10.2 Hopsworks Feature Store Migration to MongoDB

Problem: The Hopsworks free-tier Kubernetes cluster exhausted all CPU nodes after a zombie Spark job from an interrupted backfill ran for 9+ hours. The YARN ResourceManager state was corrupted by repeated kill/resubmit attempts. Kubernetes monitoring showed 0/23 nodes available with max node group size reached. Additionally, `fg.read()` returned empty data even when `fg.insert()` appeared to succeed, because the offline Hudi table was never updated when the feature group was created with `online_enabled=False`.

Fix: Migrated the feature store entirely to MongoDB Atlas M0. No Spark, no Hudi, no materialization jobs. Hopsworks is retained only for model artefact storage.

```
# Before
fg = fs.get_or_create_feature_group(...)
fg.insert(df, write_options={"wait_for_job": False})

# After
from mongo_store import store_df
store_df(df) # immediate write, immediately readable
```

10.3 LSTM seq_len=1 — No Temporal Sequence

Problem: The LSTM was built with `seq_len=1`. With sequence length 1, the LSTM cell has nothing to recur over — it is mathematically equivalent to a dense layer. The `return_sequences=True` in the first LSTM layer produced a (batch, 1, 64) tensor with no temporal structure. The model was labelled as an LSTM but gained zero benefit from recurrence.

Fix: Increased sequence length to 24 hours (one diurnal cycle), enabling the model to learn meaningful temporal dependencies.

```
SEQ_LEN = 24 # one full diurnal cycle
Xs_train, ys_train = make_sequences(X_train_s, y_train.values, SEQ_LEN)
```

10.4 Train/Serve Skew from Forecast Columns

Problem: Before, the code stored 12 forecast columns (`temp_forecast_24h`, etc.) in the feature group as NaN for all historical rows, then imputed them to their column median at training time (~30.2°C, constant). At inference, real varying forecast values were used. The model learned these features were useless (constant in training) but then encountered a varying distribution at inference — a severe train/serve skew that directly reduced 48h/72h R^2 .

Fix: All forecast columns were removed from the feature group schema. They are fetched at inference time only and never written to the feature store.

10.5 AQI Source Mismatch — PM2.5 Formula vs Official us_aqi

Problem: The original backfill computed AQI using a single-pollutant PM2.5 formula: `aqi = pm25_to_aqi(pm25)`. Open-Meteo provides `us_aqi` — the official EPA AQI computed from all pollutants (PM2.5, PM10, O3, NO2, SO2, CO). The formula produced systematically lower AQI values, creating a target mismatch between training data and live inference rows. The model was trained to predict the wrong target.

Fix: Standardized on `us_aqi` as the primary AQI target in both backfill and inference pipelines, ensuring consistency across the full system.

10.6 XGBoost Optimization Hyperparameters

Problem: The initial XGBoost configuration used default parameters: `n_estimators=500`, `learning_rate=0.05`, `max_depth=6`, no column subsampling, and no explicit regularisation. At the 1h horizon this produced a reasonable $R^2=0.987$ but with visible high-frequency noise in predictions. At the 48h and 72h horizons the model overfit on the training set. The `colsample_bytree=1.0` default meant every tree saw all 60 features, encouraging the model to split on noisy or redundant features. The `subsample=1.0` default meant no row-level bagging, reducing variance reduction.

Fix: Parameters were tuned to reduce complexity and force regularization:

Table 3: XGBoost Hyperparameter Tuning

Parameter	Before	After	Reason
<code>n_estimators</code>	500	500	Kept sufficient
<code>learning_rate</code>	0.05	0.02	Slower learning reduces overfitting
<code>max_depth</code>	6	5	Shallower trees generalise
<code>subsample</code>	1.0	0.8	Row bagging adds variance reduction
<code>colsample_bytree</code>	1.0	0.8	Feature subsampling prevents splits on redundant columns
<code>reg_alpha</code>	0	0.5	L1 regularisation encourages sparse feature use
<code>reg_lambda</code>	1.0	2.0	L2 regularisation shrinks large weights

11. Automated CI/CD Pipeline

The pipeline is fully automated using two GitHub Actions workflows triggered by cron-job.org. GitHub's native cron on the free tier delays execution by up to 60 minutes, making it unsuitable for hourly scheduling. cron-

job.org instead sends a `workflow_dispatch` POST to the GitHub API at minute 0 of every hour, triggering the runner immediately.

- **Feature pipeline (hourly):** Installs minimal dependencies, fetches current AQI and weather, engineers lag and rolling features from MongoDB history, and upserts one row. Runtime is typically under 3 minutes.
- **Training pipeline (daily, 02:00 UTC):** The training pipeline fetches all feature rows from MongoDB, retrains Ridge, RandomForest, XGBoost, and LSTM across all four forecast horizons, computes R^2 -proportional ensemble weights, and saves the full model bundle to the Hopsworks Model Registry.

12. Dashboard and Live Implementation

The Streamlit dashboard is deployed on Streamlit Cloud (Python 3.11.9). On load it downloads the latest models from Hopsworks and the most recent feature row from MongoDB, runs ensemble inference across all four horizons, and renders the results in three main sections:

1. **Current conditions** — AQI reading with colour-coded EPA category, pollutant levels (PM2.5, PM10, O3, NO2, SO2, CO), and weather metrics.
2. **3-day forecast** — three cards (Day 1, Day 2, Day 3) colour-coded by predicted AQI category.
3. **Historical chart** — 7-day AQI time series with EPA category bands.

A health alert triggers for any predicted AQI above 100, displaying the affected population group and recommended actions. A full-width emergency banner displays for predicted AQI above 300.

13. Limitations and Future Work

- **Data volume:** The training set covers only six months (Dec 2025 – Jun 2026), spanning a single winter-to-summer transition. The 15-point AQI mean shift between training (mean 94) and test (mean 79) periods directly causes negative absolute R^2 at 48h/72h. One full year of data covering both seasonal transitions would eliminate this structural bias.
- **No monsoon data:** Karachi's southwest monsoon (July–September) produces the city's cleanest air and highest wind speeds. The current models have never seen these ventilation patterns, making forecasts during this period unreliable.

14. Conclusion

The Karachi AQI Predictor demonstrates that a fully automated, production-grade air quality forecasting system can be built and operated entirely on free-tier cloud infrastructure. The system successfully ingests live data every hour, retrains daily, and serves real-time 1h to 72h AQI forecasts through a public dashboard, all without a persistent server or manual intervention. Exploratory analysis reveals a persistent Moderate pollution baseline (mean AQI = 91), a pronounced winter-to-summer seasonal decline, high within-day variability, and strong short-lag autocorrelation — all of which directly motivate the feature engineering choices. SHAP analysis confirms historical AQI state and trajectory as dominant predictors, with

meteorological variables providing meaningful secondary contributions. LIME explanations provide intuitive instance-level narratives linking current atmospheric conditions to the forecasts.